EXPRESSsuite

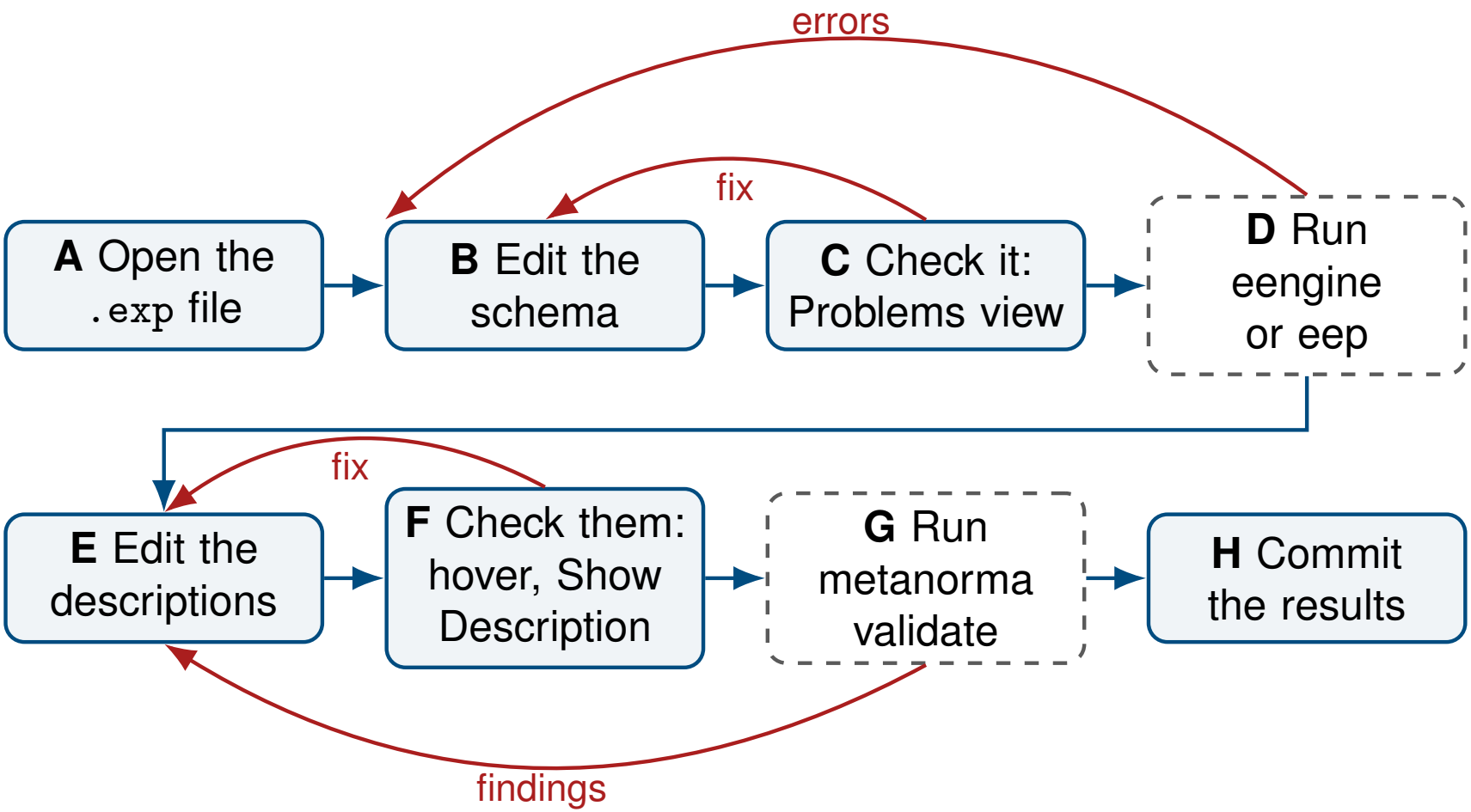
A Visual Studio Code Insiders extension for EXPRESS schemas

For STEP developers and model reviewers

November 2026

Software version 0.5.0

The generic authoring loop: edit and check in VS Code, verify outside it, then commit



Blue: in VS Code with EXPRESSsuite. Dashed: outside the editor. Red: feedback. A metanorma finding in the schema itself goes back to **B**.

Contents

1. What EXPRESSsuite is, and how to install it
2. The checks provided in the Problems view, with examples
3. Descriptions, diagrams and formulas
4. Confidential schema content
5. Where to get it, and where to report a problem

Backup

Section 1

What EXPRESSsuite is, and how to install it

It is an annotated EXPRESS native language editor. It understands the language and provides more than colouring.

While you type

- Syntax colouring for `.exp` files
- Completion of entity, type and attribute names
- Snippets for common EXPRESS patterns

While you read

- Go to Definition, Peek Definition
- Find All References
- Rename Symbol across files
- Outline of the schema (Go to Symbol)

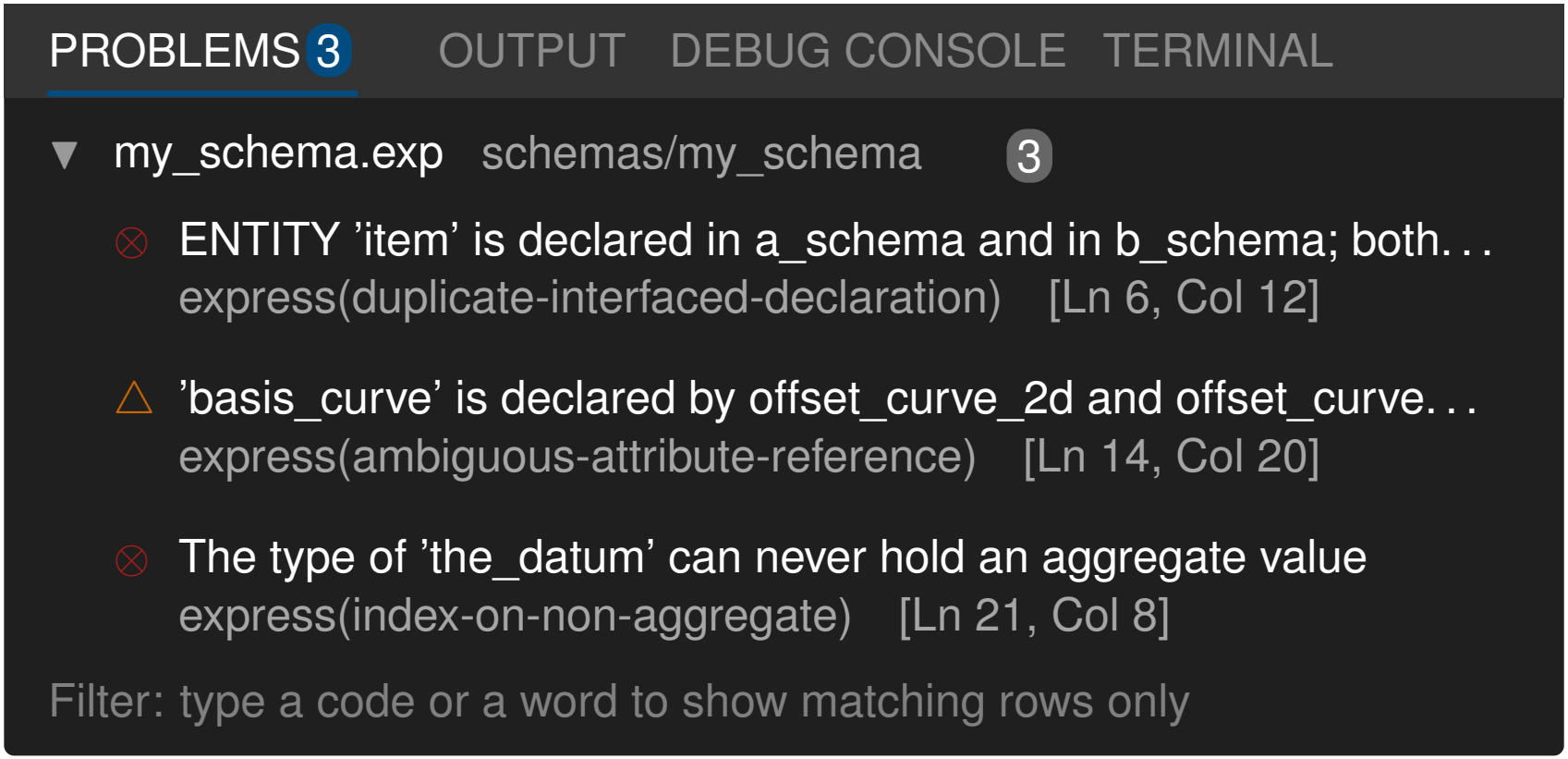
All of these follow the same name-binding rules as the checks on the next slides. A name you can jump to is a name the checker accepts.

EXPRESSsuite checks and explains EXPRESS schemas while you edit them

- EXPRESS is the schema language of ISO 10303 (ISO 10303-11:2004). Schema files end in `.exp`.
- EXPRESSsuite is an extension for Visual Studio Code (VS Code), a free code editor.
- It is tailored for ISO 10303 development: the SC4 naming conventions, the annotated EXPRESS documentation format and the wg12-step folder layout are built in.
- It reads every `.exp` file in the opened folder, so it knows how the schemas refer to each other, and reports problems in the **Problems** view.
- It renders the description of a name when you point at it.

Vocabulary: a *diagnostic* is one reported problem. A *diagnostic code* is its short name, for example `index-on-non-aggregate`.

The Problems view lists every diagnostic with its severity, message, code and line



Open it with `Ctrl+Shift+M` (macOS `Cmd+Shift+M`). Click a row to jump to the line. The filter box narrows the list to one code or one word. Rows with a light bulb offer a quick fix, for example for a name in the wrong case.

You install it from the VS Code Marketplace

1. Open VS Code. Press `Ctrl+Shift+X` (macOS: `Cmd+Shift+X`) to open the Extensions view.
2. Search for **EXPRESSsuite**. Click **Install**.
3. Open a folder that contains `.exp` files, for example a clone of the wg12-step repository.
4. Open any `.exp` file. The extension starts and reads the folder.

Works on Windows, macOS and Linux. Needs VS Code 1.140 or newer.

By default, files whose name contains `_lf.exp` or `concatenated` are loaded but not interpreted: you can open and read them, but they are not checked and their names are not indexed. They are generated copies of the module files, and interpreting them would duplicate every name. Setting: `expresssuite.excludedFiles`.

Section 2

The checks provided in the Problems view, with examples

Five checks apply clauses of ISO 10303-11:2004

Diagnostic code	Clause	What it reports
`duplicate-interfaced-declaration`	10.1 b, cl. 11	one name, two declarations in one schema scope
`ambiguous-attribute-reference`	12.7.3	`x.a` where two entities declare `a` at the same depth
`redeclaration-not-specialization`	9.2.3.4, 9.2.7	a redeclared attribute widens the original type
`index-on-non-aggregate`	12.6.1	`x[1]` where `x` can never be an aggregate
`reference-statement-missing`	cl. 11	a name is used but not interfaced

A *linking error* is also reported when a name cannot be found at all. The next slides show one example of each check.

Four checks apply SC4, TF1 and WG11 conventions

Diagnostic code	Source	What it reports
`declaration-name-case`	SC4 rule	an ARM entity in lowercase, or a MIM name in uppercase; a quick fix is offered
`duplicate-loop-index`	13.9.1	a `LOCAL` variable redeclares a `REPEAT` index
`informal-proposition-*`	TF1 rule	an `IPn` annotation without its `-IPn:` signature
`redundant-declaration`	WG11 N239	an identical repeated definition in a long form (warning)

These rules are not in ISO 10303-11. They are SC4, TF1 and WG11 conventions, applied while editing.

One name with two declarations is reported in every schema that sees both

Clause 11 defines what a schema *sees*: its own declarations, plus what `USE FROM` and `REFERENCE FROM` bring in. In one scope, a name may denote one declaration only (10.1 b).

```
SCHEMA a_schema;  ENTITY item; END_ENTITY;  END_SCHEMA;
SCHEMA b_schema;  ENTITY item; END_ENTITY;  END_SCHEMA;

SCHEMA my_schema;
  USE FROM a_schema;
  USE FROM b_schema;    -- item now arrives twice
END_SCHEMA;
```

```
duplicate-interfaced-declaration:  ENTITY 'item' is declared in
a_schema and in b_schema; both are visible in my_schema
```

The report is placed on the `USE FROM` line that brought the second copy.

An attribute reference looks only in the entity, its supertypes and its subtypes

ISO 10303-11:2004 12.7.3: for `x.a`, the attribute `a` is searched in the declared type of `x`, then in its supertypes and subtypes, nearest first. Never in a sibling entity.

```
ENTITY curve; END_ENTITY;
ENTITY offset_curve_2d SUBTYPE OF (curve);
  basis_curve : curve; END_ENTITY;
ENTITY offset_curve_3d SUBTYPE OF (curve);
  basis_curve : curve; END_ENTITY;

-- somewhere: c : curve;  ...  c.basis_curve
```

```
ambiguous-attribute-reference (warning):  'basis_curve' is
declared by offset_curve_2d and offset_curve_3d at the same depth
```

Write `c\offset_curve_2d.basis_curve` to say which one you mean.

A redeclared attribute must keep or narrow the original type

ISO 10303-11:2004 9.2.3.4 a) with 9.2.7: the new type shall be the same as, or a specialization of, the original type. `INTEGER` narrows `REAL`; `SET OF x` narrows `BAG OF x`; a subtype narrows its supertype.

```
ENTITY measured;
  value : REAL;
END_ENTITY;

ENTITY counted SUBTYPE OF (measured);
  SELF\measured.value : INTEGER;      -- accepted: narrower
END_ENTITY;

ENTITY wrong SUBTYPE OF (measured);
  SELF\measured.value : STRING;       -- reported
END_ENTITY;
```

```
redeclaration-not-specialization:  STRING is not a specialization
of REAL
```

Indexing a value that can never be an aggregate is an error

ISO 10303-11:2004 12.6.1: the operand of `[ ]` shall be an aggregate value (a `LIST`, `SET`, `BAG` or `ARRAY`). This slip, taken from a draft of ISO 10303-47, was not reported before 0.5.0.

```
ENTITY datum;
  identification : STRING;
END_ENTITY;

ENTITY holder;
  the_datum : datum;              -- one instance, not a list
WHERE
  WR1: the_datum[1]\datum.identification <> 'x';
END_ENTITY;
```

```
index-on-non-aggregate:  the type of 'the_datum' can never hold
an aggregate value
```

Letter case follows the SC4 convention: ARM capitalised, MIM lowercase

ISO 10303-11 gives letter case no meaning. SC4 schemas do: the ARM entity is `Product`, the MIM entity is `product`, and they are two different entities. EXPRESSsuite applies this project rule (rulings of 2026-10-29 and 2026-10-30):

- Entity names and schema names bind only to the exact spelling. `SUBTYPE OF (mesh)` does not name `ENTITY Mesh`.
- Type, attribute, function, rule and enumeration names bind regardless of case. `x : Unit_kind` names `TYPE unit_kind`.
- In a long form (schema name ends in `_lf`) every name binds regardless of case, as WG11 N239 requires.

The published SMRL v8 follows the convention without exception: 3,460 of 3,460 ARM entity names begin with an uppercase letter, 1,359 of 1,359 MIM entity names with a lowercase one.

A MIM reference written in ARM spelling is reported as not found

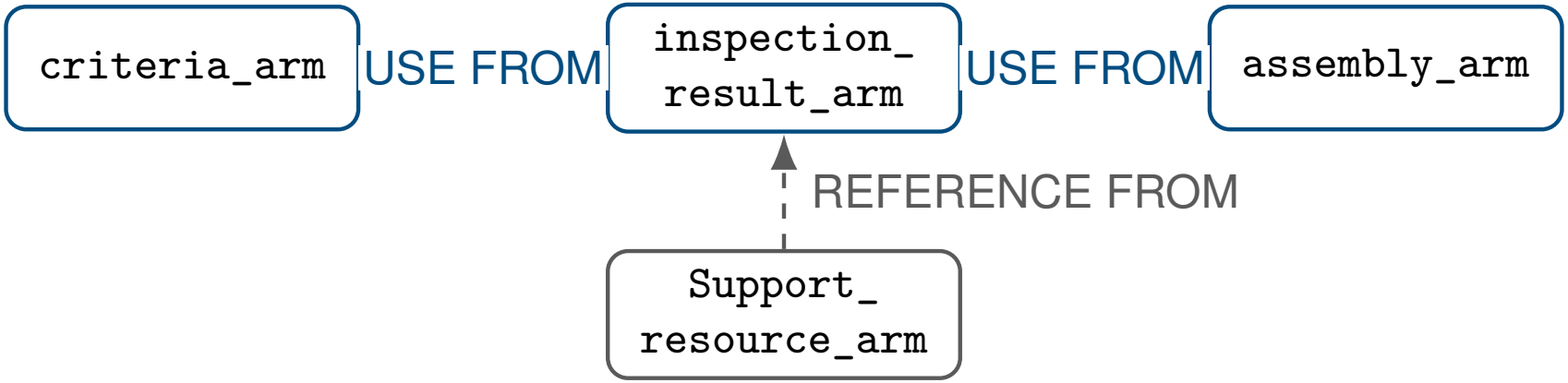
In the published assembly MIM, one `DERIVE` line writes the supertype in ARM spelling. The 67 other references in the same file are lowercase.

```
ENTITY a3m_equivalence_criterion_for_assembly
  SUBTYPE OF (a3m_equivalence_criterion);
  SELF\a3m_equivalence_criterion.measured_data_type : ...;
DERIVE
  SELF\A3m_equivalence_criterion.accuracy_types : LIST [0:0] OF ...;
END_ENTITY;
```

```
The entity 'A3m_equivalence_criterion' could not be found in the
supertype/subtype of 'a3m_equivalence_criterion_for_assembly'
```

The fix is one letter. A message that names the case difference is planned (see the backup slides).

Interfaces follow clause 11: USE FROM passes names on, REFERENCE FROM does not



- The assembly ARM sees every declaration of the criteria ARM, two steps away, because `USE FROM` re-exports what it imports (11.1, 11.3).
- The assembly ARM does *not* see `Support_resource_arm`: a `REFERENCE FROM` is never passed on (11.2, 11.3).
- Item lists and renames are honoured. One declaration reached by two routes counts once.

Short names: `criteria_arm` is `Annotated_3d_model_equivalence_criteria_arm`, and so on.

Two type properties are proved: specialization and indexability

Check	Clause	What is proved
redeclared attribute type	9.2.3.4, 9.2.7	`spec_sound`: if the relation accepts the new type, every value of the new type is a value of the original type
index on a non-aggregate	12.6.1	`notIndexable_sound`: if the predicate holds for a type, no value of that type can be indexed, as a binary, a string or an aggregate

Both are proved against a transcription of eengine's value semantics (`TypeP`), so "value" means what the reference validator means by it.

Two scope properties are proved: 12.7.3 visibility and clause 11 scope

Check	Clause	What is proved
reference scope (`x.a`, `x\e`)	12.7.3, 12.7.4	`mem_visible_iff`, `resolve_unique`, `resolve_ambiguous`: the walk finds exactly the 12.7.3 set; ambiguity exactly when two entities at the nearest depth declare the name
schema scope and clashes	10.1 b, cl. 11	`iterate_sound`, `export_complete`, `clash_iff`: the loop computes exactly the clause 11 scope; a clash exactly when one name denotes two declarations

In Lean, `sorry` marks a proof left open. The folder contains none: 78 theorems, 93 build-checked facts, the three standard axioms only.

Section 3

Descriptions, diagrams and formulas

Point at a name to read its rendered description

SC4 schemas carry their documentation inside the `.exp` file, as *annotated EXPRESS*: `(*"schema.entity" ... *)` blocks after `END_SCHEMA`. EXPRESSsuite renders them.

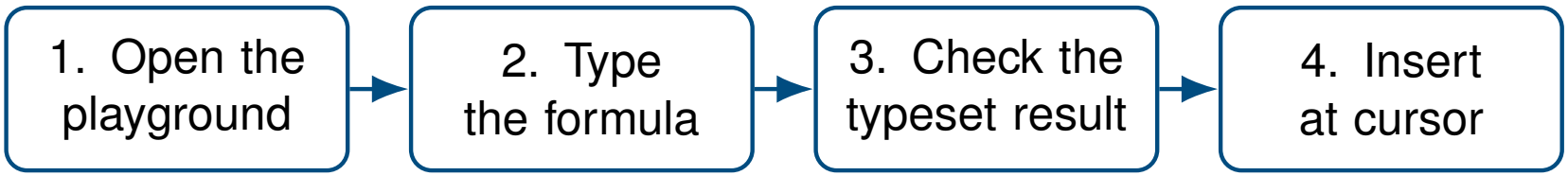
- **Hover**: move the mouse over a name. A popup shows its description, with cross-references as links and formulas as pictures.
- **Show Description** (right-click): the full description opens in a panel beside the editor. Click a link in it to open the description of that name.
- The description is found through the schema's interfaces, so hovering `string_literal` in a schema that references it shows the owning schema's text.

Formulas written as `stem:[...]` (AsciiMath) or `latexmath:[...]` are typeset by Plurimath, the same converter Metanorma uses to publish the standards.

EXPRESS-G diagrams open from the schema and link back to the source

- **Show EXPRESS-G Diagram** (right-click): opens the `expgN.svg` diagram that contains the entity at the cursor. If several diagrams match, a list is offered.
- Click any box in the diagram. The editor jumps to that entity's declaration.
- **Detect Duplicate Declarations in Schema Closure**: the same clause 11 check as the live diagnostic, on demand, for the schema at the cursor.

Write a formula in the playground, check it, then insert it into the schema



1. Command Palette (`Ctrl+Shift+P`, macOS `Cmd+Shift+P`): **EXPRESSsuite: Open AsciiMath Playground**. A panel with two panes opens.
2. Type AsciiMath in the left pane, for example `sum_(i=1)^n i^3`.
3. The right pane shows the formula typeset. It updates when you stop typing.
4. Place the cursor in the description in the editor, then click **Insert at cursor**. The playground writes `stem:[...]` there.

The other direction: select an existing `stem:[...]` in the editor, right-click, **Send Selection to AsciiMath Playground**, and edit it there.

Section 4

Confidential schema content

EXPRESSsuite itself sends nothing anywhere

- The extension makes **no network calls**. There is no telemetry, no update check, no download at run time.
- Every panel (description, diagram, playground) is configured with the browser rule `connect-src 'none'`, so a panel cannot open a connection even if it tried.
- The source code contains no `fetch` and no `http` import. You can check this yourself: the code is public.
- All rendering, including formulas, runs on your computer.

This matters for schemas under development that are not yet public.

VS Code itself ships GitHub Copilot Chat, which does send text out

Copilot Chat is built into VS Code. Its right-click entries (*Explain*, *Add File to Chat*, *Open Inline Chat*, *Review*) send the selected text, the surrounding lines and parts of other open files to Microsoft. Starting VS Code with `-disable-extensions` does not remove them.

For schemas under non-disclosure, do one of these:

1. Sign out of GitHub in that VS Code window (account icon, bottom left). Without a login, Copilot Chat cannot reach its service.
2. Or set `"chat.disableAIFeatures": true` in the workspace settings file `.vscode/settings.json`.
3. Best practice: keep one separate, signed-out VS Code window for confidential schemas.

Section 5

Where to get it, and where to report a problem

Where to get it, and where to report a problem

- Marketplace: search **EXPRESSsuite** in VS Code, or open `marketplace.visualstudio.com/items?itemName=TRThurman.expresssuite`
- Source, release notes and issues: `github.com/TRThurman/EXPRESSsuite`
- To report a problem: open an issue with the schema text that shows it and the message from the Problems view. The issue form asks for both.
- Licence: derived from NIST easyEXPRESS (Sylvere Krima, Allison Barnard Feeney, Rosemary Astheimer). NIST software is public domain in the United States. Modifications are listed in `NOTICE.md`.

Thomas Thurman, `thomas@trthurman.consulting`

Backup

Tool maintenance

how every change is tested, and the known limits of 0.5.0

Every change is tested against the 1,410 published SMRL v8 schema files

1,410	1,287	103
`.exp` files	loaded as one workspace	long forms, built alone

- Every diagnostic on every file is recorded in a baseline file in the repository.
- A code change that moves one diagnostic shows up as a reviewable difference.
- The same result is produced on macOS, Linux and Windows, and on repeated runs in one process.
- The baseline records behaviour, not correctness. Known false positives are listed next to it, so they are not mistaken for schema defects.

Known limits in 0.5.0

- Long forms and concatenated files are loaded but not interpreted by default. Interpreted together with the modules, they would duplicate every name.
- Some attribute references inside `QUERY` expressions, function results and long attribute paths are not typed yet. They are reported as linking errors. These sites are listed with the baseline as known false positives.
- The letter-case rule is stricter than ISO 10303-11. A reference written in the wrong case is reported as "could not be found". A clearer message is planned.
- The EXPRESS-G viewer needs the `expgN.svg` files beside the schema, as wg12-step provides them.

EXPRESSsuite uses Lean 4 the way AWS uses it for Cedar: an executable model that the code must match

- Lean 4 is a programming language and a proof assistant in one. A definition written in Lean runs, and a theorem about it is checked line by line by a small kernel, the only part that has to be trusted.
- AWS builds its Cedar authorization language this way: the model of each component is written in Lean, its properties are proved, the product is written in Rust, and no version is released unless the model, the proofs and the tests that compare the two are current. The models are about ten times smaller than the product.

Sources: AWS Open Source Blog, “Lean Into Verified Software Development”, 2024; lean-lang.org/use-cases/cedar. Full list on the “Lean sources” slide.

Four of the nine checks are built from an executable Lean 4 model that the code must match

The method is the one AWS uses for its Cedar language (previous slide). The four checks are those whose code holds 81 % of the decision points of all nine (McCabe cyclomatic complexity; the table is on the next slide).

- **Model:** the clause semantics of each check are written in Lean and proved: the specialization relation (9.2.3.4), the indexability predicate (12.6.1), the 12.7.3 visibility walk, and the clause 11 scope with its clashes (10.1 b).
- **Product:** the TypeScript checker transcribes those definitions clause by clause and cites each one at the function that implements it.
- **Agreement:** the standard's own examples are build-checked facts in Lean, the unit tests exercise the same cases in TypeScript, and the SMRL v8 baseline pins the result on 1,390 published schemas.

The four proved checks hold 81 % of the decision points of all nine checks

Check	Lean proof	Complexity
redeclaration-not-specialization	yes	157
duplicate-interfaced-declaration (and redundant-declaration)	yes	139
index-on-non-aggregate	yes	138
informal-proposition-*	no	54
reference-statement-missing	no	30
ambiguous-attribute-reference	yes	15
declaration-name-case	no	14
duplicate-loop-index	no	7
proved checks		449 of 554

McCabe cyclomatic complexity per check: its validator method plus the module that implements its relation, measured with ESLint's `complexity` rule on 0.5.0. The resolved-type builder (216) serves the two type checks and is split between them.

Where the model lives, and what it does not cover

- The folder `lean/` in the repository holds the definitions of the four checks and the proofs about them. `lake build` checks every proof; it is run by hand, not yet by the extension's continuous integration.
- The TypeScript code of the checker is a line-by-line transcription of those definitions, kept in step with them.
- The standard's own examples (9.2.3.4 EXAMPLE 2 and 3, 11.3 EXAMPLE 2) are build-checked facts.
- Two mistakes in the first version of the redeclaration rule were found by these examples, not by users: `SET OF x` narrows `BAG OF x`, and `INTEGER` narrows `REAL` (9.2.7).
- The other five checks (name case, loop index, informal propositions, missing reference statements, redundant repeats) are tested, not proved.

Lean sources

- K. Hietala, E. Torlak, “Lean Into Verified Software Development”, AWS Open Source Blog, 8 April 2024. `aws.amazon.com/blogs/opensource/lean-into-verified-software-development/`
- “Lean Powers Secure Software at AWS: Cedar’s Journey with Verified Development”, `lean-lang.org/use-cases/cedar/`
- J. W. Cutler et al., “Cedar: A New Language for Expressive, Fast, Safe, and Analyzable Authorization”, arXiv:2403.04651, 2024.
- Cedar security documentation, `docs.cedarpolicy.com/other/security.html`
- M. de Medeiros et al., “Verified Foundations for Differential Privacy” (SampCert), arXiv:2412.01671, 2024. `reservoir.lean-lang.org/@leanprover/sampcert`
- “Scaling Verification of Cryptographic Software with Aeneas, Rust, and Lean”, arXiv:2609.15648, 2026; Microsoft Research, “Verifying Rust cryptography in SymCrypt, from standards to code”.
- Galois, “Real-time Robotics Control in the Lean Language”, 29 March 2021.
- Lean homepage and use cases, `lean-lang.org`; Software Verification in Lean workshop, 2026.